

QUDA MultiGrid 的算法理解与 PyQCU 复现

MATPC 紧凑 odd Schur、P/R、Clover/Gauge 与旧实现对照

代码审计与代数验证

PyQCU · Lattice QCD

2026 年 8 月 29 日

本轮结论：MATPC compact odd Schur 已接入单 rank CUDA 生产路径

已完成

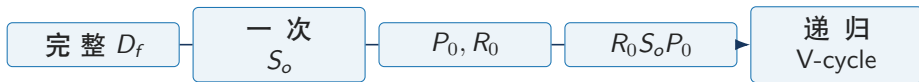
新增 CompactParityLayout、CompactParityOperator；
setup_operator="schur"
首层构造 odd Schur，后续层直接 Galerkin 粗化。

本轮证据

定向命令与 examples/pyqcu 均为 **26 passed**；compact 多层 full residual 1.02×10^{-7} ，独立 Schur 解为 1.12×10^{-7} 。

边界

旧 solver.multigrid 保留；单 rank CUDA 的 setup/solve 已实测，但不宣称 MPI、混合精度或 QUDA 原生 kernel 性能等价。



阶段判断： QUDA MATPC 的核心约束已落成可测不变量：奇偶布局、局部块逆、 S_o 严格伴随、 RS_oP 层级和 full 解重构均有回归证据。

来源：实现：pyqcu/solver/_quda_multigrid.py:1094-1292,1373-2331；测试：examples/pyqcu/test_quda_multigrid.py:434-725；命令：pytest-qexamples/pyqcu。

研究问题与验收标准：先对齐代数，再谈加速

问题	本轮可检验定义	状态
QUDA 的层级如何递归？	每层由 $B \rightarrow V \rightarrow (P, R) \rightarrow D_c = RDP$ ，运行期再接 smoother、restrict、coarse solve、prolong。	确证
Clover/Gauge 是否被保留？	local Clover 进入 X ，Gauge/hopping 进入 $Y^\pm$ ；不能只粗化 hopping。	确证
parity 是否只在最细层？	MATPC 首层形成 S_o ，后续在 odd 几何上继续 RS_oP ，不能重复减半。	确证
PyQCU 是否可复现？	compact/full round-trip、Schur adjoint、重构、Galerkin、求解和 QCU 资产 shape 全部回归。	确证
性能是否等价？	仍需 MPI、混合精度、大格 setup 及 QUDA 原生 kernel 同口径 benchmark；本轮只报告单 rank 实测。	未验证

判定口径

命令/恒等式为“确证”，控制流对应为“推断”；MPI、混合精度和 QUDA 原生对照仍为“未验证”，单 rank 证据不外推为 scaling。

来源：对照：QUDA multigrid/transfer/coarse_op/dirac_coarse；日志：logs/v20260829.txt。

CUDA 单 rank 生产回归通过：两层有收益，三层保持正确但代价更高

配置	层	MG solve (s)	full res.	MG/ref
$8 \times 8 \times 8 \times 16$	2	0.159	6.88×10^{-7}	1.72×10^{-5}
$8 \times 16 \times 16 \times 16$	2	0.234	2.41×10^{-7}	2.40×10^{-6}
$8 \times 16 \times 16 \times 16$	3	0.694	3.23×10^{-7}	4.36×10^{-6}

可复现设置

$m = 0.05$, complex64, $E = (12, 48[, 48])$, restart = 10, 单 MPI rank;
C++ 内部 full-op true residual 依次为 8.37×10^{-7} 、 1.33×10^{-7} 、 3.53×10^{-7} 。

来源：命令：source./env.sh&&bash./build.sh; pythonexamples/qcu/conftest.schur.multigrid.py; 结果：logs/schur_mg_results.json。

直接证据

后端重建	SUCCESS
两层小格	2.369×
两层大格	2.005×
三层大格	PASS ; 0.455×

判读

full-opres. 是 Python Wilson/Clover full operator 对 C++ MG 输出的重算；MG/ref 是相对 C++ BiStabCG 参考解。三层结果证明递归接线正确，但当前粗解配置不支持性能外推。

QUDA 的递归控制流: reset 建层, operator 做一次 V-cycle

输入: D_ℓ , B_ℓ , geometry, precision

reset: 创建/刷新 B_ℓ 与 Transfer_ℓ

$B_{\ell+1,i} \leftarrow R_\ell B_{\ell,i}$ (若不在各层重新生成)

createCoarseDirac: 构造 coarse operator

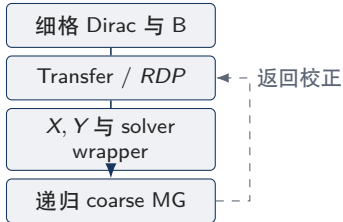
区分 residual / smoother

createCoarseSolver: 封装 solver + MG

operator: prepare $\rightarrow$ pre-smooth $\rightarrow Rr \rightarrow$ recurse

$\rightarrow Pe_c \rightarrow$ post-smooth

输出: full 或 MATPC solution, 并按需要 reconstruct



关键分离

QUDA 同时维护 residual operator、smoother operator 及 sloppy precision; 它们必须在 solution type 不一致时显式重构并重算 residual。

来源: QUDA: multigrid.cpp:72-197; PyQCU: _quda_multigrid.py:1915-1986。

P/R 的本质: aggregate 几何 map 加 fine-spin 到 coarse-spin map

几何与自由度

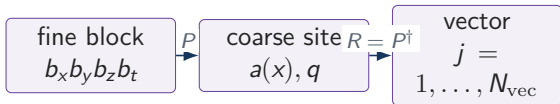
$$\begin{aligned}a(x) &= (\lfloor x_0/b_0 \rfloor, \dots, \lfloor x_3/b_3 \rfloor), \\ q(s) &= \lfloor s/b_s \rfloor \quad (\text{Wilson/Clover}), \\ D_f &= 4 \times 3 = 12, \quad D_c = 2N_{\text{vec}}.\end{aligned}$$

对 staggered, $b_s = 0$, coarse spin 由 site parity 承载。

延拓与限制

$$\begin{aligned}(P\eta)_{sc}(x) &= \sum_j V_{scqj}(x) \eta_{qj}(a(x)), \\ (R\psi)_{qj}(a) &= \sum_{x \in a, s, c} V_{scqj}^*(x) \psi_{sc}(x), \\ R &= P^\dagger, \quad RP \simeq I_c.\end{aligned}$$

V 只在同一 aggregate、同一 coarse-spin block 内正交。



极限检查: 若 N_{vec} 超过 aggregate 局部维数, 局部 CGS 无法产生独立列; 容量约束是构建前硬条件。

来源: QUDA: transfer.cpp:200-328; PyQCU: QudaTransfer。

块 CGS：局部正交化决定 $R = P^\dagger$ 是否数值可靠

for each aggregate a and coarse spin q :

$v_j \leftarrow B_j$ (first pass), $v_j \leftarrow V_j$ (later pass)

$h_{ij} = \langle v_i, v_j \rangle_{a,q}$

$v_j \leftarrow v_j - \sum_{i < j} h_{ij} v_i$

$v_j \leftarrow v_j / \|v_j\|_{a,q}$

repeat $n_{\text{block_ortho}}$ passes and save V

局部维数

Wilson/Clover: $N_{\text{vec}} \leq N_c |a| b_s$; staggered: 每个 coarse parity 只拥有 $N_c |a| / 2$ 个位置。

来源: QUDA: `block_orthogonalize.cuh`:149-239; PyQCU: `_quda_multigrid.py`:189-219, 319-355; 直测: `QudaMultigrid.diagnostics()`。

PyQCU

`_cg_orthogonalise()` 按 block 批量执行重复 CGS; QudaTransfer 以相同的 V 定义 prolong/restrict。

当前直测

compact 两层例的最大 block Gram 误差: 4.34×10^{-7} ; RP 相对误差: 1.67×10^{-7} 。

粗算子不是“只剩 dslash”：local Clover 与 Gauge hopping 都在 RDP

Galerkin 定义

$$\begin{aligned} D_{\ell+1} &= R_\ell D_\ell P_\ell, \\ (D_c x)(q) &= X(q)x(q) \\ &\quad + \sum_{\mu=0}^3 \left[Y_\mu^+(q)x(q + \hat{\mu}) \right. \\ &\quad \left. + Y_\mu^-(q)x(q - \hat{\mu}) \right]. \end{aligned}$$

X 是目标点 local block； $Y^\pm$ 是带方向和边界约定的 hopping block。

物理一致性

丢掉 X 会改变 Schur 补，丢掉 Y 会改变传播；二者共同组成粗 Dirac operator，不能把粗层误写成不完备的 hopping-only dslash。

来源：QUDA: coarse_op.cuh; PyQCU: _quda_multigrid.py:664-910。

代码审计与代数验证

对象	实现语义
细层 Gauge	U_μ 进入 Wilson hopping；细层 D_f 由 dslash 提供。
细层 Clover	C 进入 local M ，参与 diagonal inverse。
QUDA Y	UV/VUV 累加 forward/backward link；必要时保留 halo。
QUDA X	coarse local Clover/identity； coarse-on-coarse 继续投影。
PyQCU	materialize 逐列探测 RDP；lazy 执行 $R(D(Px))$ 。

预条件粗算子：先构造 X^{-1} ，再形成 $Yhat$ 与 Schur

局部预条件

$$D_c = X + H, \quad D_{pc} = X^{-1}(D_c - X) \\ = X^{-1}H,$$

$$D_{full,pc} = X^{-1}D_c = I + D_{pc},$$

$$Yhat_{\mu}^{+}(q) = X^{-1}(q)Y_{\mu}^{+}(q).$$

backward link 存在存储位置平移，并带有伴随和右侧 $X^{-\dagger}$ 的约定。

来源：QUDA: `coarse_op_preconditioned.cuh`; PyQCU: `_quda_multigrid.py`:912-1056。

DiracCoarsePC::Dslash: 只施加 D_{pc}
DiracCoarsePC::M: even/odd Schur action
prepare/reconstruct: full 与 MATPC
solution 转换
solution type 不一致：先 reconstruct，再重算 residual
PyQCU: `preconditioned_apply` →
`preconditioned_full_apply`

实现检查

backward $Yhat$ 的索引、滚动方向和 $X^{-\dagger}$ 右因子必须同时正确；只验证 forward link 不足以证明 PC 算子正确。

奇偶块代数: MATPC 的目标是 odd Schur, 不是半截 full field

块矩阵推导

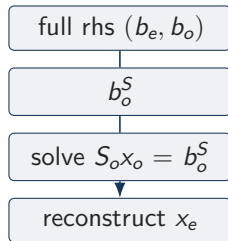
$$D = \begin{pmatrix} D_{ee} & D_{eo} \\ D_{oe} & D_{oo} \end{pmatrix}, \quad S_o = D_{oo} - D_{oe} D_{ee}^{-1} D_{eo}.$$

$$b_o^S = b_o - D_{oe} D_{ee}^{-1} b_e,$$

$$x_e = D_{ee}^{-1} (b_e - D_{eo} x_o).$$

local diagonal D_{ee} 是 Clover/identity block, 必须可逆。

来源: QUDA: `multigrid.cpp:1131-1224`, `dirac_coarse.cpp:506-626`; PyQCU: `_quda_multigrid.py:1295-1370`。



目标 parity

本轮固定保留 odd、消去 even; even-even 只需交换 parity 选择, 公式结构不变。

compact parity 布局: T 轴配对依赖空间 parity

定义

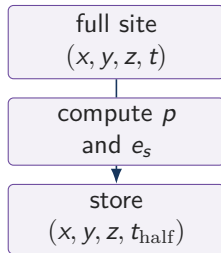
完整场为 $[dof, X, Y, Z, T]$, 紧凑 parity 场为 $[dof, X, Y, Z, T/2]$ 。对空间 parity $e_s = (x + y + z) \bmod 2$:

$$p = (x + y + z + t) \bmod 2,$$
$$t_{\text{full}} = 2t_{\text{half}} + ((p - x - y - z) \bmod 2).$$

两个具体分支

$$\begin{aligned} \text{even } p = 0 & \quad t_{\text{full}} = 2t_{\text{half}} + e_s \\ \text{odd } p = 1 & \quad t_{\text{full}} = 2t_{\text{half}} + (1 - e_s) \end{aligned}$$

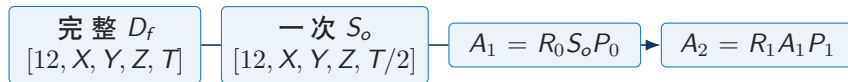
来源: QCU 对照: `cpp/cuda/qcu/src/multigrid.cu:1579-1743`; PyQCU: `_quda_multigrid.py:1094-1161`; 测试: `test_quda_multigrid.py:434-458`。



硬约束

T 必须为偶数; 奇子格不是简单的“奇数 t ”切片。该依赖若错一位, P/R 和每次 V-cycle 的残差都会失真。

新层级的关键决定：只做一次 finest Schur，粗层不再二次 checkerboard



为什么不能再做 Schur

S_o 已经只作用在 odd compact geometry。若对 A_1 再调用旧 checkerboard Schur，会把已有 $T/2$ 几何再次减半，得到错误的 $T/4$ 层级；QUDA 的递归 MATPC 路径应直接 coarsen 当前算子。

来源：QUDA: multigrid.cpp:347-434; PyQCU: _quda_multigrid.py:1691-1753; compact 测试: test_quda_multigrid.py。

自由度协议

full 模式的 `dof_list=[12,24,24]` 表示总 DOF，每个 coarse-spin block 用 $24/2 = 12$ 个向量；compact 模式条目直接作为当前 coarse DOF。

compact Schur 的实现闭环: action、严格伴随、rhs 与 reconstruct

接口	计算链	结果布局
apply	embed target odd $\rightarrow D_{oe}$; 局部 D_{ee}^{-1} ; 再取 D_{eo} , 最后减去 diagonal	$[dof, X, Y, Z, T/2]$
adjoint_apply	用完整 $D^\dagger$ 取相反块, 并使用 $(D_{ee}^{-1})^\dagger$; 不假定 S_o Hermitian	同上
rhs	$b_o - D_{oe} D_{ee}^{-1} b_e$	odd compact rhs
reconstruct	$x_e = D_{ee}^{-1}(b_e - D_{eo} x_o)$, 再嵌回 full	$[dof, X, Y, Z, T]$

严格伴随的必要性

若 setup 选择 $D^\dagger D$, 自定义 fine operator 必须提供 fine_adjoint; Wilson/Clover 则自动使用 $D^\dagger = \gamma_5 D \gamma_5$ 。这是 setup 与 Schur adjoint 共用的同一物理对称性。

来源: PyQCU: _quda_multigrid.py:579-661, 1164-1292; 测试: test_quda_multigrid.py:460-505。

setup 与求解: compact 只改变算子空间, 不改变外层验证语义

random: 直接使用随机/显式 B

inverse/null: 求 $A\delta = AB$, $B \leftarrow B - \delta$

test: 从零初值求 $AB_{\text{new}} = B$

cg/ca-cg: 默认 $A = D^\dagger D$

krylov/gcr: 默认直接作用于非厄米 D

compact 首层: $A = S_o$;

后续层: $A = RS_oP$ (当前 coarse op)

外层 solve

$$\begin{aligned} b &\rightarrow b_o^S, \\ x_o &\leftarrow \text{FGMRES}(S_o, \text{V-cycle}), \\ x &\rightarrow \text{reconstruct}(b, x_o). \end{aligned}$$

公共 apply 仍返回 full D ; 另有
apply_compact 返回 S_o 。

验证

TEST_VECTOR_SETUP 对 $D = 2I$ 得到精确的
 $B/2$; compact setup history 显式记录首层
operator 为 Schur。

来源: PyQCU: _quda_multigrid.py:1728-1903; 测试: test_quda_multigrid.py:506-525。

V-cycle 的 compact 语义: coarse solve 直接作用在当前 Galerkin 算子

$$r_\ell = b_\ell - A_\ell x_\ell$$

$$x_\ell \leftarrow \text{MR}(A_\ell, r_\ell)$$

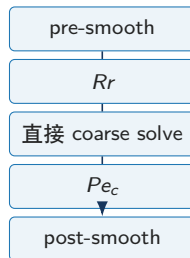
$$r_{\ell+1} \leftarrow R_\ell r_\ell$$

$$e_{\ell+1} \leftarrow \text{solve}(A_{\ell+1}, r_{\ell+1})$$

$$x_\ell \leftarrow x_\ell + P_\ell e_{\ell+1}$$

$$x_\ell \leftarrow \text{MR}(A_\ell, r_\ell, x_\ell)$$

$$K(r_\ell) = x_\ell \quad \text{作为 FGMRES 右预条件器}$$



防止重复减半

compact 分支的 smoother 与底层 solve 直接调用 `operator.apply`; 不再访问旧 `ParitySchurOperator`。

来源: PyQCU: `_quda_multigrid.py`:2053-2133; QUDA 运行期对照: `multigrid.cpp`:1131-1224。

QCU 过渡资产: blocked P/R 与 33 点 stencil 都携带 compact 元数据

P/R ABI

$V_{\text{Python}} : [s, c, q, j, X, Y, Z, T],$

$V_{\text{QCU}} : [E, e, X_c, b_x, Y_c, b_y, Z_c, b_z, T_c, b_t],$

$E = N_{\text{coarse spin}} N_{\text{vec}}, \quad e = N_{\text{fine spin}} N_{\text{fine color}}.$

`to_qcu_blocked()` 只做轴重排和 aggregate 拆分; R 的共轭不被写入 P 。

资产元数据

`compact_parity`、`parity`、`eliminated_parity`、`fine_full_shape`、`fine/coarse_dof` 和 `operator_kind`。

来源: PyQCU: `_quda_multigrid.py`:446-470,817-910,2135-2191; QCU 对照: `cpp/cuda/qcu/src/multigrid.cu`:858-874。

33 点槽位

`sitting` $(0, 0, 0, 0)$

`hop_nn` $\pm \hat{\mu}, \mu = 0, \dots, 3$

`hop_diag` $\pm \hat{\mu} \pm \hat{\nu}$, 六个 pair

`extent = 2` 的重复邻居平均分到两个符号槽; `extent = 1` 折入 `sitting`; `strict` 模式拒绝更宽支撑。

本轮 compact 标识

`level=0`: `compact_schur`; 后续 `level`: `compact_galerkin`。这明确禁止把 `coarse transition` 当成二次 Schur。

旧实现与新实现并存：新路径补齐语义，不静默替换历史基线

维度	旧 solver.multigrid	新 solver.QudaMultigrid
MG-0 parity 粗层算子	保留已有 fine/最细层 parity 行为 历史路径的粗层未完整表达 full dslash，作为对照	首层显式 compact odd S_o 每层 $D_{l+1} = R_l D_l P_l$ ，支持 lazy/materialize
粗层 parity	未延伸为完整递归 compact MATPC	后续层直接在 compact 几何上 Galerkin，不二次减半
Clover/Gauge	依赖旧接口与历史约定	细层复用 Wilson/Clover；local X 和 hopping Y 均保留
P/R	历史扁平 E 约定	显式 coarse spin、局部 CGS、 $R = P^\dagger$ 和 QCU blocked 导出
角色	历史/生产实验基线，不能删除	QUDA 语义参考与迁移验证基线

兼容策略

pyqcu/solver/__init__.py 同时导出旧类与 QudaMultigrid、CompactParityLayout、CompactParityOperator；测试明确断言新旧导出不是同一类。

来源：旧实现：pyqcu/solver/_multigrid.py；新实现：pyqcu/solver/_quda_multigrid.py；导出：pyqcu/solver/__init__.py:1-14；测试：test_quda_multigrid.py:101-104。

本轮代数证据：布局、Schur、Galerkin 与求解均通过

指标 (4^4 , complex64)	实测
compact RP 相对误差	1.67×10^{-7}
transfer 伴随相对误差	6.76×10^{-9}
compact block Gram 最大误差	4.34×10^{-7}
RS_oP Galerkin 相对误差	7.81×10^{-8}
Schur 严格伴随内积误差	5.45×10^{-9}
eliminated-even 重构误差	2.18×10^{-8}

compact 求解/资产	实测
MG full residual	1.02×10^{-7}
direct Schur residual	1.12×10^{-7}
S_o 与 dslash parity action	0
operator DOF	12, 2, 2
asset kind	Schur / Galerkin
目录级回归	26/26

解释

compact projection residual 不要求任意随机 full field 被 $PP^\dagger$ 精确表示；本轮验收的是 RP 、adjoint、Galerkin 和 full residual。

证据含义

左侧数字证明 Python 参考实现的代数和布局约定；新增 CUDA 回归页证明 QCU 单 rank 的生产 setup/solve 与 full-op residual。两者都不等价于 MPI scaling 或 QUDA 原生数值路径的性能结论。

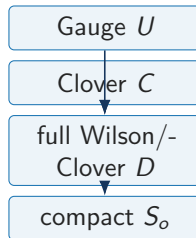
来源：命令：定向测试与 examples/pyqcu 均 26 passed；数值：compact 直测与 examples/qcu/conftest.schur.multigrid.py。

测试覆盖已扩展到物理输入：Wilson、Clover 与 Gauge 均进入 compact setup

测试矩阵

输入	验证内容
identity diagonal	TEST setup 的 $S = 2l$ 精确缩放
Wilson + $U = I$	parity action = compact Schur
Wilson + random Gauge	setup / RDP / reconstruct
Wilson + Clover C	local inverse / Schur reconstruct
两层 compact hierarchy	solve / QCU asset shape

来源：测试：test_quda_multigrid.py:694-725；细层接口：_quda_multigrid.py:1490-1531。



直接结果

同一测试函数对 `clover_term=None` 与显式 local Clover 均通过 `transfer_RP`、`galerkin_RDP` 和 `schur_reconstruct_even`。

当前限制与风险：代数闭环不等于生产性能闭环

项目	当前状态	影响与下一步
coarse materialize	单 rank QCU 已接通；逐列 Python asset setup 为 331.7–972.5 s	资产生成仍是瓶颈，下一步用 batch/MT/local build。
compact Schur	Python action/adjoint 与单 rank QCU full residual 均已验证	MPI、混合精度和跨 rank parity 仍未验证。
33 点支撑	QCU stencil 有 strict guard	超出 nearest/diagonal 支撑时必须改 kernel 或保留 matrix-free，不能静默截断。
精度/通信	本轮单 rank、complex64	混合精度、ghost、MPI 拓扑和 scaling 未覆盖。
性能结论	单 rank C++ solve：两层 2.005–2.369×，三层 0.455×	不外推 QUDA 吞吐、MPI scaling 或通用 setup 收益。

风险控制：单 rank QCU 的 compact 接线和 full residual 已确证；MPI、混合精度、QUDA 原生对照仍保持“未验证”，并以 compact layout、Schur、RDP 和真残差作为后续 golden contract。

来源：实现边界：_quda_multigrid.py:748–910,2145–2163；QCU：examples/qcu/conftest.schur.multigrid.py；历史证据：examples/qcu/dev87/。

阶段结论与后续：单 rank QCU 已闭环，下一步进入 MPI 与 setup 优化

已确认

- ▶ QUDA 层级主线为 $B \rightarrow V \rightarrow P/R \rightarrow RDP$;
- ▶ X 、 Y 、 X^{-1} 、 $Yhat$ 和 parity 是同一粗算子体系;
- ▶ compact MATPC 只在 finest 做一次 odd Schur。

本轮交付

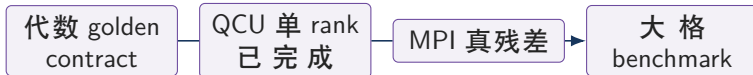
新类、导出、26 项回归、compact QCU asset metadata、单 rank CUDA 2/3 层回归，以及本报告 `..._5.tex/.pdf`。

旧路径

`pyqcu/solver/_multigrid.py` 保留为缺陷/历史对照，不做无声替换。

下一阶段验收

1. 2/4 rank full residual + parity round-trip;
2. 混合精度、ghost 与 MPI 拓扑回归;
3. 批量/多线程 coarse asset setup 优化;
4. QUDA 原生 kernel 同口径对照。



来源：行动依据：代数测试、`logs/schur_mg_results.json` 与 `examples/qcu/confctest.schur.multigrid.py`; MPI/资源节点待排期。

附录：对象—源码—证据索引

对象	QUDA 对照	PyQCU 当前证据
递归 MG geometry/spin map	multigrid.cpp:72-197, 1131-1224 transfer.cpp:200-257	QudaMultigrid.setup/v_cycle QudaTransfer, 含 Wilson/staggered
P/R	transfer.cpp:259-328	局部 CGS、 $R = P^\dagger$ 、QCU blocked export
coarse X/Y PC/Yhat	coarse_op.cuh:1035-1459 dirac_coarse.cpp:351-430, 506-626	materialized/lazy RDP、33 点打包 preconditioned_apply 与 local inverse
parity	multigrid.cpp:1131-1224	ParitySchurOperator 与 compact Schur
compact map	QCU multigrid.cu:1579-1743	CompactParityLayout 与 round-trip 测试
旧实现	PyQCU 历史路径	solver/_multigrid.py 未替换

证据等级

确证：本轮命令或数值直测；**推断**：源码控制流和代数推导的一致性判断；**未验证**：尚未运行的 MPI、混合精度、跨 rank parity 或 QUDA 原生对照项目；单 rank QCU setup/solve 已有直接证据。

来源：工作目录：/root/PyQCU；对照目录：/root/PyQCU/refer/git-rep/quda。

附录：最小调用与 compact API

```
1 from pyqcu.solver import QudaMultigrid
2
3 mg = QudaMultigrid(
4     U=U, clover_term=C, kappa=kappa,
5     null_vectors=[B0, B1],
6     block_size=[(2,2,2,2), (1,1,1,1)],
7     max_level=3, use_parity=True,
8     setup_operator="schur")
9 x = mg.solve(b)
10 Sbo = mg.apply_compact(bo)
11 assets = mg.qcu_transition_assets()
12
```

来源：接口：pyqcu/solver/_quda_multigrid.py:1400-1544, 2021-2034, 2135-2191。

输入约定

b: full $[4, 3, X, Y, Z, T]$ 或 flat $[12, X, Y, Z, T]$; apply_compact: odd $[12, X, Y, Z, T/2]$; assets: 每条 transition 的 blocked P 与 stencil。

注意

T 必须偶数；自定义 operator 启用 normal setup 时须显式提供 fine_adjoint 和 parity 所需 fine_diagonal。

附录：本轮验证命令与交付闸门

```
$ pytest -q examples/pyqcu/test_quda_multigrid.py    26 passed in 8.22s
$ pytest -q examples/pyqcu    26 passed in 8.99s
$ python -m py_compile ...    通过
$ source ./env.sh && bash ./build.sh    CUDA backend SUCCESS
$ python examples/qcu/conftest.schur.multigrid.py    2L OK/PASS; 3L PASS
$ git diff --check    通过
$ xelatex ... (two passes)    Overfull=0, Float too large=0
$ pdfinfo / pdftoppm    页数一致，逐页渲染检查通过
```

交付判定

页面数、渲染页数和逐页检查数必须一致；报告中的“确证”只对应上表、CUDA 生产回归或前述可复查源码/恒等式证据。MPI、混合精度和 QUDA 原生 benchmark 在本轮保持未验证。

来源：文件：docs/report_quda_multigrid_reproduction_20260829_5.tex/.pdf；Git 收尾将在报告编译与渲染检查后执行。